تقرير التقدّم 4: النتائج الأولية للجهتين

ملخّص كل الشغل من البداية، ونتائج القياس على بيانات الأوراق البحثية لقسمي Code vs Doc و Code vs Code

التاريخ: 29-10-2026 الموديل: poolside/laguna-s-2.1:free البيئة: حاوية ai_web (Docker)

0.44	0.55	0.35	169+
كشف التناقضات بالتوثيق (CASCADE نفسها: 0.21)	PFP: التنبيه بيروح لما يتصلّح التوثيق (القديم: 0.00)	كشف التعديلات يلي بتكسر الكود (القديم: 0.22)	تعديل بيكسر انكشف وما كان القديم يشوفه

1. الفكرة باختصار

المشروع فيه قسمين لكشف المشاكل:

- **Code vs Doc:** هل التوثيق بيوصف الكود صح؟
- **Code vs Code:** بين نسختين من الكود، شو تغيّر؟ وهل التغيير بيكسر يلي عم يستعملوا الكود؟

عم نحسّن القسمين خطوة خطوة حسب خطط مبنية على أوراق بحثية (DocPrism و METAMON للتوثيق، و AexPy و NPM و Safer Builders للكود). وبعد كل خطوة منقيس على **بيانات الأوراق نفسها**، مش على بيانات عملناها نحنا. وبتوجيه الدكتور، اخترنا **ورقة وحدة لكل قسم**: CASCADE للتوثيق، و NPM للكود.

2. الخطوات يلي خلصت

الخطوة	شو عملنا	ليش
0	تجميد النسخة القديمة، وأداة قياس ثابتة	لحتى نقارن القديم بالجديد بعدل. النسخة القديمة محفوظة وما بتنلمس (منتأكد منها ببصمة)، وكل طلبات الذكاء الاصطناعي ثابتة (temperature 0، seed 42، موديل واحد مثبّت)
1	معلومات كاملة عن كل دالة	صار النظام يقرأ الجسم والمعاملات والقيم الافتراضية ونوع الإرجاع والتوثيق الداخلي والـ async والـ constructors. قبل كان عنده الاسم والتوقيع بس
2	تعديل بنية البيانات	تصنيفات DocPrism (تناقض مباشر، وعد زيادة، نقص بالتوثيق) ومستويات الخطر. وصلّحنا خطأ كان يوقّف كل Code vs Code
3	تقسيم التوثيق وربط كل جزء بدالته	بدل ما ينبعت التوثيق كله دفعة وحدة، كل دالة بتنفحص مع الجزء يلي بيحكي عنها
3ب	الدوال يلي برّا الكلاسات	كان النظام ما يشوفها أبداً، وهي نص بيانات DocPrism وكتير منتشرة بـ JavaScript
4	سؤال جديد للذكاء الاصطناعي	3 أسئلة محددة متل DocPrism، مع طلب نسخ الجملة من التوثيق والسطر من الكود كما هم، ودرجة ثقة

5	محلّل التوثيق الجديد	إعادة المحاولة، ورمي أي جواب جملته مش موجودة فعلاً، وعتبة ثقة 0.7، وإخفاء "النقص بالتوثيق" عن المستخدم
6	ربط المحلّل بالنظام	صار يشتغل تلقائياً. وشلنا الفحص القديم للتوقيع يلي كان سبب كل الإنذارات الكاذبة، وصار "غير موثّق" إحصائية مش تنبيه

كل خطوة إلها اختبارات وتقرير بـ `core_ai/tests/reports/` . المجموع 97 اختبار، وكلهم ناجحين.

3. Code vs Doc على بيانات CASCADE

البيانات والمقياس

بيانات CASCADE فيها 71 دالة جافا توثيقها متناقض مع الكود، ومع كل وحدة **نفس الدالة بعد ما تصلّح التوثيق**. قسّمناها: 25 زوج للتطوير (منفتح أخطاءه)، و46 زوج للاختبار (أرقام بس، بالمحطات المهمة).

PFP هو أهم مقياس هون: من التناقضات يلي انكشفت، بكم حالة التنبيه راح لما تصلّح التوثيق؟ إذا التنبيه ضل، معناها النظام عم ينبّه لسبب ما إله علاقة بالتناقض.

النتائج على قسم التطوير (25 زوج = 50 حالة)

النظام	Precision	Recall	F1	Specificity	PFP	TP	FP	FN	TN
القديم	0.50	0.68 (وهمية)	0.58	0.32	0.00	17	17	8	8
الحالي بدون ذكاء اصطناعي	—	0.00	—	**1.00**	—	0	0	25	25
الحالي مع الذكاء الاصطناعي	**0.58**	0.44	0.50	0.68	**0.55**	11	8	14	17

مقارنة مع الأرقام المنشورة بورقة CASCADE

النظام	Precision	Recall	Specificity
أداة CASCADE	0.88	0.21	0.97
LLM-A: سؤال مباشر للموديل	0.59–0.55	0.46–0.29	0.81–0.64
LLM-S: سؤال أبسط	0.56–0.53	0.67–0.54	0.58–0.42
نظامنا (قسم التطوير)	0.58	0.44	0.68

شو بيعني هاد

- **القديم كان "بيكشف" بالصدفة.** نبّه على 17 نسخة متناقضة، و**نفس العدد بالزبط** من النسخ المصلّحة، يعني PFP = 0. والسبب خطأ بقراءة توقيعات الجافا.
- **الجديد بيكشف تقريباً ضعف أداة CASCADE**، ودقّته بنفس مستوى أفضل نتيجة ذكاء اصطناعي بالورقة.
- **وبأكتر من نص الحالات يلي كشفها، التنبيه راح لما تصلّح التوثيق.** يعني تنبيهاته فعلاً إلها علاقة بالتناقض.

يلي لسا ناقص

- 8 إنذارات كاذبة من 25 حالة سليمة (Specificity = 0.68). منفتحها ومنشوف سببها بالمرحلة الجاية.
- الأرقام على 25 زوج بس، وأرقام الورقة على كل الـ 71. وقسم الاختبار لسا ما انقاس مع الذكاء الاصطناعي، لأنه بدّه 92 طلب (يومين من الحد المجاني).
- عينة وحدة ما اشتغل عليها الذكاء الاصطناعي لأنه خلص الحد اليومي، فاشتغل عليها الفحص الثابت بس.

4. Code vs Code على بيانات NPM

البيانات

- **ورقة NPM (النسخة 3، مفتوحة على Zenodo):** فيها 1519 تغيير بيكسر الكود، موسومين يدوياً من المؤلفين بـ 131 مشروع JavaScript/TypeScript. ثلثينهم تقريباً تغييرات **سلوكية** (نفس التوقيع بس الشغل تغيّر)، والباقي **بنيوية** (حذف، إعادة تسمية، تغيير توقيع).
- **الكود:** الورقة ما فيها الكود نفسه، فنزّلناه من GitHub لكل تعديل، قبل وبعد. نزل كود **1460** تعديل من 1487.
- **أمثلة سليمة:** الورقة فيها بس تعديلات بتكسر، فما في طريقة نقيس فيها الإنذارات الكاذبة. لهيك جبنا **338 تعديل عادي** من نفس المشاريع ونفس الفترة، رسالتهم ما فيها أي إشارة لكسر. هدول **"سليمين على الأرجح"**، مش موسومين من المؤلفين.
- **التقسيم:** 20% تطوير و 80% اختبار، ثابت (seed 42).

طريقة القياس

- **الوحدة هي التعديل كله**، لأنه وسم الورقة على مستوى التعديل مش الملف.
- لكل ملف كود تغيّر، منقارن النسخة قبل وبعد. والتعديل بينحسب "منبَّه عليه" إذا طلع فيه تغيير بيكسر أو إعادة تسمية أو تغيير سلوكي.
- ملفات الاختبارات والأمثلة منتجاهلها، لأنها ما بتأثر على يلي عم يستعملوا المكتبة. والتعديلات يلي ما ضل فيها ملف ينفحص بتطلع من المقاييس (165 تعديل).
- **هالقياس بدون ذكاء اصطناعي**، يعني الفحص البنيوي بس، وما كلّف ولا طلب.

النتائج

المجموعة	النظام	Precision	Recall	Specificity	كشف البنيوي	كشف السلوكي	TP	FP	FN	TN
تطوير	القديم	0.85	0.19	0.82	0.40	0.10	51	9	215	40
تطوير	**الحالي**	0.85	**0.30**	0.71	**0.53**	0.20	79	14	187	35
اختبار	القديم	0.91	0.22	0.89	0.39	0.14	246	24	851	200
اختبار	**الحالي**	0.90	**0.35**	0.82	**0.55**	0.25	385	41	712	183

الكشف حسب نوع التغيير (كل البيانات)

نوع التغيير بالورقة	العدد	القديم	الحالي
تغيير سلوكي	922	126	223
تغيير التوقيع	130	44	78
حذف دالة	117	52	69
إعادة تسمية دالة	47	15	26

حذف حقل	27	14	17
إعادة تسمية حقل	27	8	11
حذف كلاس	21	12	13
إعادة تسمية كلاس	20	18	18
حذف ملف كامل	15	1	1
نقل دالة	12	2	2

التعديل الواحد ممكن يكون فيه أكتر من نوع، فالمجموع أكبر من عدد التعديلات.

شو بيعني هاد

- **الحالي لقى 169 تعديل بيكسر ما كان القديم يشوفهم، وخسر 2 بس.** السبب الأساسي الخطوة 3ب: صار النظام يشوف الدوال يلي برّا الكلاسات، وهي الشكل الأشيع بـ `JavaScript`.
- بالتغييرات البنيوية، الكشف طلع من 0.39 لـ 0.55.

يلي لسا ناقص

- **الثمن:** 22 إنذار كاذب زيادة، فنزلت الـ Specificity من 0.89 لـ 0.82. وأكتر سبب إلها إنه النظام بيعتبر **أي تغيير بالتوقيع** كسر، حتى إضافة معامل اختياري. وهاد بالزبط يلي بتصلّحه **الخطوة 7**.
- **التغييرات السلوكية ما بتنكشف فعلياً بدون ذكاء اصطناعي.** يلي انكشف منها انكشف لأنه صادف في تغيير بنيوي بنفس التعديل. وهون دور الخطوات 8 لـ 11.
- **حذف ملف كامل ما بينكشف:** الأداة حالياً بتتجاهل الملفات المحذوفة كلها.

ملاحظة على الـ Precision

الـ Precision طالعة عالية (0.85 لـ 0.91)، بس هي **مضخّمة**، لأنه 83% من العينات بتكسر الكود. فالرقم الأصدق للإنذارات الكاذبة هو الـ Specificity.

5. الأدوات يلي انبنت للقياس

الملف	شو بيعمل
`evaluation/harness/evaluate.py`	قياس Code vs Doc للنظامين، مع حساب كل المقاييس والـ PFP، وحفظ ردود الذكاء الاصطناعي لحتى إعادة القياس ما تكلّف
`evaluation/harness/evaluate_npm.py` (جديد)	قياس Code vs Code على NPM على مستوى التعديل، مع الكشف حسب نوع التغيير
`evaluation/paper_sets/build_paper_sets.py`	بيبني عينات CASCADE و DocPrism من البيانات الأصلية
`evaluation/paper_sets/fetch_npm_code.py`	بينزّل كود كل تعديل بورقة NPM من GitHub، قبل وبعد
`evaluation/paper_sets/fetch_npm_negatives.py` (جديد)	بينزّل التعديلات العادية من نفس المشاريع
`evaluation/paper_sets/PROGRESS.md`	جدول التقدّم بعد كل خطوة

6. الخطوات الجاية

1. **بكرا:** قياس Code vs Code مع الذكاء الاصطناعي على حوالي 25 تعديل من قسم التطوير. منختار تعديلات فيها ملفات قليلة، لأنه كل ملف بياخد طلب.
2. **الخطوة 7:** مقارنة توقيعات أذكى، متل إنه إضافة معامل اختياري ما بتكسر. المتوقّع إنها تقلّل الإنذارات الكاذبة بـ NPM.
3. **الخطوات 8 لـ 11:** مقارنة الأجسام، وتصنيف التغيير، ومستوى الخطر، وما منبعت للذكاء الاصطناعي غير يلي فعلاً تغيّر جسمه.
4. نفتح الـ 8 إنذارات الكاذبة بـ Code vs Doc ومنشوف سببها.
5. بالمحطات المهمة، منقيس على قسم الاختبار تبع CASCADE مع الذكاء الاصطناعي.

كل الشغل ضمن الحد المجاني (50 طلب باليوم). النتائج التفصيلية لكل حالة بـ `evaluation/paper_sets/results/` .